

## SQL Aggregation Functions – Practical Task

**Scenario: Online Learning Platform**

**Duration: 2 Days**

### Introduction

You are working with an online learning platform similar to Coursera or Udemy, where:

- Students enroll in online courses
- Each course belongs to a category and is taught by an instructor
- Students track their progress and give ratings

You will use SQL aggregation functions to analyze data and answer real-life business questions.

### Important Instructions (Read First)

1. You must create the tables and insert the sample data first
2. Do NOT change the table structure or data
3. All tasks use SELECT queries only
4. Test your queries whenever possible

### Step 1: Create Tables

```
CREATE TABLE Instructors (  
    InstructorID INT PRIMARY KEY,  
    FullName VARCHAR(100),  
    Email VARCHAR(100),  
    JoinDate DATE  
);
```

```
CREATE TABLE Categories (  
    CategoryID INT PRIMARY KEY,  
    CategoryName VARCHAR(50)  
);
```

```
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    Title VARCHAR(100),  
    InstructorID INT,  
    CategoryID INT,  
    Price DECIMAL(6,2),  
    PublishDate DATE,  
    FOREIGN KEY (InstructorID) REFERENCES Instructors(InstructorID),  
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)  
);
```

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FullName VARCHAR(100),  
    Email VARCHAR(100),  
    JoinDate DATE  
);
```

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,
```

```
CourseID INT,  
EnrollDate DATE,  
CompletionPercent INT,  
Rating INT CHECK (Rating BETWEEN 1 AND 5),  
FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);
```

### ◆ Step 2: Insert Sample Data

**INSERT INTO Instructors VALUES**

```
(1, 'Sarah Ahmed', 'sarah@learnhub.com', '2023-01-10'),  
(2, 'Mohammed Al-Busaidi', 'mo@learnhub.com', '2023-05-21');
```

**INSERT INTO Categories VALUES**

```
(1, 'Web Development'),  
(2, 'Data Science'),  
(3, 'Business');
```

**INSERT INTO Courses VALUES**

```
(101, 'HTML & CSS Basics', 1, 1, 29.99, '2023-02-01'),  
(102, 'Python for Data Analysis', 2, 2, 49.99, '2023-03-15'),  
(103, 'Excel for Business', 2, 3, 19.99, '2023-04-10'),  
(104, 'JavaScript Advanced', 1, 1, 39.99, '2023-05-01');
```

**INSERT INTO Students VALUES**

```
(201, 'Ali Salim', 'ali@student.com', '2023-04-01'),
```

```
(202, 'Layla Nasser', 'layla@student.com', '2023-04-05'),  
(203, 'Ahmed Said', 'ahmed@student.com', '2023-04-10');
```

**INSERT INTO Enrollments VALUES**

```
(1, 201, 101, '2023-04-10', 100, 5),  
(2, 202, 102, '2023-04-15', 80, 4),  
(3, 203, 101, '2023-04-20', 90, 4),  
(4, 201, 102, '2023-04-22', 50, 3),  
(5, 202, 103, '2023-04-25', 70, 4),  
(6, 203, 104, '2023-04-28', 30, 2),  
(7, 201, 104, '2023-05-01', 60, 3);
```

**SQL Execution Order (Reference)**

**SQL executes queries in this order:**

**FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**

- **WHERE** filters rows
- **HAVING** filters aggregated results

## **Day 1 – Core Aggregation Practice**

### **Objectives**

- **Understand aggregation functions**
- **Use GROUP BY correctly**
- **Read aggregated results**

### **◆ Part 1: Warm-Up**

1. **Display all courses with prices.**

2. Display all students with join dates.
3. Show all enrollments with completion percent and rating.
4. Count instructors who joined in 2023.
5. Count students who joined in April 2023.

#### ◆ Part 2: Beginner Aggregation

1. Count total number of students.
2. Count total number of enrollments.
3. Find average rating per course.
4. Count courses per instructor.
5. Count courses per category.
6. Count students enrolled in each course.
7. Find average course price per category.
8. Find maximum course price.
9. Find min, max, and average rating per course.
10. Count how many students gave rating = 5.

#### ◆ Part 3: Extended Beginner Practice

11. Count enrollments per month.
12. Find average course price overall.
13. Count students per join month.
14. Count ratings per value (1–5).
15. Find courses that never received rating = 5.
16. Count courses priced above 30.
17. Find average completion percentage.
18. Find course with lowest average rating.

## **Reflection (End of Day 1)**

**Answer briefly:**

- **What was easiest?**
- **What was hardest?**
- **What does GROUP BY do in your own words?**

## **Day 1 Mini Challenge**

### **Course Performance Snapshot**

**Show:**

- **Course title**
- **Total enrollments**
- **Average rating**
- **Average completion %**

## **Day 2 – JOIN + Aggregation + Analysis**

### **Objectives**

- **Use JOIN with aggregation**
- **Apply HAVING correctly**
- **Think like a data analyst**

## **◆ Part 4: JOIN + Aggregation**

1. **Course title + instructor name + enrollments.**
2. **Category name + total courses + average price.**
3. **Instructor name + average course rating.**
4. **Student name + total courses enrolled.**
5. **Category name + total enrollments.**

6. Instructor name + total revenue.
7. Course title + % of students completed 100%.

#### ◆ Part 5: HAVING Practice

! Use HAVING only.

1. Courses with more than 2 enrollments.
2. Instructors with average rating above 4.
3. Courses with average completion below 60%.
4. Categories with more than 1 course.
5. Students enrolled in at least 2 courses.

#### ◆ Part 6: Analytical Thinking

Answer using SQL + short explanation:

1. Best performing course.
2. Instructor to promote.
3. Highest revenue category.
4. Do expensive courses have better ratings?
5. Do cheaper courses have higher completion?

#### Final Challenge – Mini Analytics Report

1. Top 3 courses by revenue.
2. Instructor with most enrollments.
3. Course with lowest completion rate.
4. Category with highest average rating.
5. Student enrolled in most courses.

Submit:

- **SQL queries**
- **Output**
- **1–2 line explanation**

 **Expected Outcome**

**By completing this task, you will be able to:**

- **Use aggregation functions confidently**
- **Apply GROUP BY & HAVING correctly**
- **Combine JOINS with aggregation**
- **Analyze real-life data using SQL**